Northern University of Business and Technology Khulna
Department of Computer Science and Engineering



RESUMIND

AI-Powered Resume Analyzer and Career Copilot

Deployment

Live deployment links, hosting setup and secret management

Course	CSE4204 — Mobile Computing Lab
Team	CSE4204-8A-T05 (Section 8A)
Submission	Week 11 · Deployment
Members	Mahammad Hasan (11220320828) Shanawaz Sakib (11220320916) Ashikur Rahman Himel (11220320833) Rasel Ratul (11220320827)
Live app	https://ai-resume-analyzer-psi-lime.vercel.app
Repository	https://github.com/mahamud894/AI_Resume_Analyzer

Submitted to

Md. Riaz Mahmud

Assistant Professor, Department of CSE, NUBTK

Submission date: 22 October 2026

1. Live Deployment Links

RESUMIND is deployed and publicly accessible. Scan a code with a phone camera or open the links below.

Live application
https://ai-resume-analyzer-psi-lime.vercel.app

Source code (GitHub)
https://github.com/mahamud894/AI_Resume_Analyzer

Table 1. *Submitted links*

Item	URL
Live application (frontend)	https://ai-resume-analyzer-psi-lime.vercel.app
Backend API	https://resumind-api-3kpd.onrender.com/api
API health check	https://resumind-api-3kpd.onrender.com/api/health
GitHub repository	https://github.com/mahamud894/AI_Resume_Analyzer
CI status (GitHub Actions)	https://github.com/mahamud894/AI_Resume_Analyzer/actions

> **Please note:** The backend runs on Render's free plan and sleeps after 15 minutes without traffic. The first request after that takes about 50 seconds; afterwards it responds normally. Open the health-check link once before testing.

1.1 Demo account

Examiners can create their own account in a few seconds (Sign Up or Sign in with Google), or use the team's demo account shared during the presentation. No personal data is required.

2. Hosting Architecture

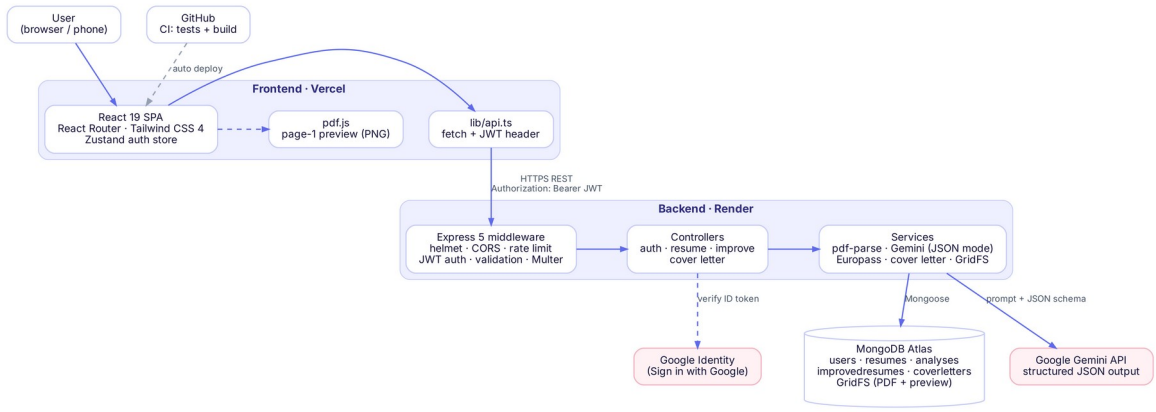


Figure 1. *Deployment architecture*

Table 2. *Deployment requirements from the handbook and how they are met*

Requirement	Our deployment
Frontend on Vercel / Netlify	Vercel — React Router app built as a static SPA (`build/client`), root directory `frontend`, configured by `frontend/vercel.json`
Backend on Render / Railway	Render web service `resumind-api` from the Blueprint `render.yaml`: root `backend`, build `npm ci --omit=dev`, start `npm start`, health check `/api/health`, Node 24
Database	MongoDB Atlas M0 cluster — collections + GridFS for uploaded files (Render's disk is temporary)
Environment variables in `.env`	Local: `backend/.env` and `frontend/.env` (git-ignored, templates in `.env.example`). Production: Vercel and Render dashboards
Automatic deployment	Every push to `main` → GitHub Actions runs tests → Vercel and Render redeploy the same commit

3. Environment Variables and Secrets

No secret is stored in the repository. Only the variable **names** are listed here; the values exist only in the hosting dashboards and in each developer's local `.env` file.

Table 3. *Backend (Render)*

Variable	Purpose	Where the value comes from
`MONGO_URI`	MongoDB Atlas connection string	Atlas → Connect → Drivers
`JWT_SECRET`	Signs login tokens	Generated automatically by Render (generateValue)
`GEMINI_API_KEY`	Google Gemini access	Google AI Studio
`GOOGLE_CLIENT_ID`	Verifies Google Sign-In tokens	Google Cloud Console → OAuth client
`CLIENT_URL`	CORS allow-list (the Vercel URL)	https://ai-resume-analyzer-psi-lime.vercel.app
`NODE_ENV · TRUST_PROXY`	production · 1 (correct client IP behind Render's proxy for rate limits)	render.yaml

Table 4. *Frontend (Vercel)*

Variable	Purpose	Value
`VITE_API_URL`	Address of the backend API	https://resumind-api-3kpd.onrender.com
`VITE_GOOGLE_CLIENT_ID`	Shows the Sign in with Google button	Same OAuth client id as the backend

3.1 Security measures for the deployment

- `.env` files are listed in `.gitignore`; only `.env.example` templates with empty values are committed.
- The Gemini key and database password exist only on the server; the browser receives only the public API URL and the public Google client id.
- CORS accepts requests only from the production frontend URL.
- HTTPS everywhere (Vercel and Render provide TLS certificates automatically).
- Atlas allows `0.0.0.0/0` because Render has no fixed IP; access is still protected by a strong database user password.

4. Deployment Procedure

4.1 Backend → Render

1. Render → New → Blueprint → select the GitHub repository; Render reads `render.yaml`.
2. Enter MONGO_URI, GEMINI_API_KEY, GOOGLE_CLIENT_ID and CLIENT_URL; JWT_SECRET is generated.
3. Deploy and check that `/api/health` returns `{"status":"ok"}`.

4.2 Frontend → Vercel

1. Vercel → Add New → Project → import the repository → Root Directory `frontend`.
2. Add VITE_API_URL (the Render URL, no trailing slash) and VITE_GOOGLE_CLIENT_ID.
3. Deploy, then set CLIENT_URL on Render to the Vercel URL and add the URL to the Google OAuth client's authorised JavaScript origins.

4.3 Continuous integration and delivery

```
git push origin main
   │
   ├─▶ GitHub Actions: backend  → npm ci → npm test (48 tests)
   │                   frontend → typecheck → build
   ├─▶ Render  : npm ci --omit=dev → npm start → health check → live
   └─▶ Vercel  : react-router build (SPA) → live
```

5. Deployment Verification

Table 5. *Checks performed on the live system*

#	Check	Result
1	Live URL opens the landing page over HTTPS on desktop and phone	Pass
2	/api/health returns status ok	Pass
3	Sign up, log in, log out, Sign in with Google	Pass
4	Upload PDF → analysis result shown	Pass
5	Europass CV, before/after score, cover letter, PDF downloads	Pass
6	Uploaded files still available after a backend redeploy (GridFS)	Pass
7	Request from another origin is blocked by CORS	Pass
8	Unknown page shows the 404 page	Pass
9	CI green on the deployed commit	Pass

5.1 Keeping the demo stable

- Wake the backend by opening /api/health 2–3 minutes before the presentation.
- Keep a resume PDF and a job description ready on the presentation laptop.
- Prepare a demo account that already contains one finished analysis, CV and cover letter.
- Keep backup screenshots (in the presentation slides) in case the internet or Gemini is unavailable.